

Course Name: Cryptography and Cyber Security

Course Code: 22CSE163

Module 2: Modern Cryptography

Dr. Deepak D J

Associate Professor

Dept. of CSE, BNMIT

Introduction to Cryptography

- The study of **mathematical techniques** for **securing digital information**, systems, and distributed computations against adversarial attacks.
- Ex: mechanisms for **ensuring integrity**, techniques for **exchanging secret keys**, protocols for authenticating users, electronic auctions and elections, digital cash, and more.
- **Classical: Security through obscurity**
- **Modern** : The security is not based on the secrecy of the protocol details but **based on sound mathematical and computational principles**.

Classical Vs Modern Cryptography

- Classical cryptography was **restricted to military**.
- Classical cryptography was mostly about **secret communication**.
- The **communication is secure** as long as the **encoding algorithm is a secret**.
- Disadvantages: Reverse engineering, coding algorithm leaks.
- Modern cryptography is **influences almost everyone**.
- Modern cryptography **breaks out of the “design-break-design”** cycle model of classical cryptography.
- The security is not based on the secrecy of the protocol details but **based on sound mathematical and computational principles**.
- **Provable security**: mathematically demonstrate that a cryptographic system is secure against a specific set of attacks.

Keys and Kerckhoffs' principle.

- “The cipher method must not be required to be secret, and it must be able to fall into the hands of the enemy without inconvenience”.
- Kerckhoffs' principle demands that **security rely solely on secrecy of the key.** – **cryptographic designs** must be made completely **public**.
- It is simply **unrealistic to assume that the encryption algorithm will remain secret.**
- **Easier to change a key than to replace an encryption scheme.**
- Easier for users to all rely on the same encryption algorithm/software (with different keys) than for everyone to use their own custom algorithm.
- “security by obscurity” - keeping algorithms secret improves security.
- Published designs undergo **public review** and are therefore likely to be **stronger**.

AES Requirements

- Private key symmetric block cipher
- 128-bit data, 128/192/256-bit keys
- Stronger & faster than Triple-DES
 - DES 168 bit key (56×3)
 - Slow in software 16×3 rounds + key expansion!
 - Designed for 70's hardware
- NIST have released all submissions & unclassified analyses
- Both C & Java implementations
- Rijndael cipher was made as AES in Oct-2000

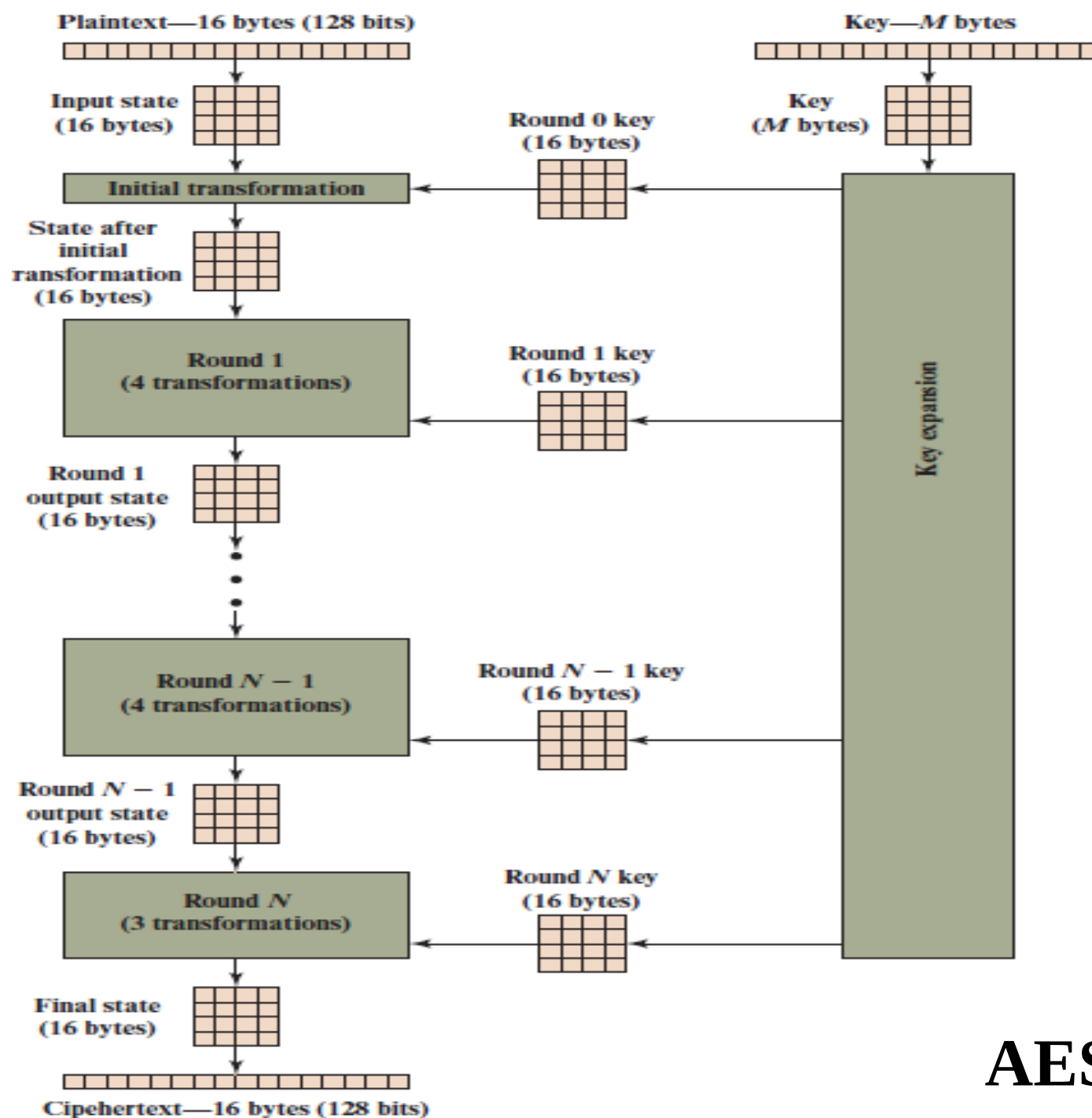
The AES Cipher - Rijndael

- Designed by Vincent Rijmen, Joan Daemen in Belgium
- Has 128/192/256 bit keys, 128 bit data
- An iterative rather than feistel cipher
 - processes data as block of 4 columns of 4 bytes
 - operates on entire data block in every round
- Designed to be:
 - resistant against known attacks
 - speed and code compactness on many CPUs
 - design simplicity

AES Rounds

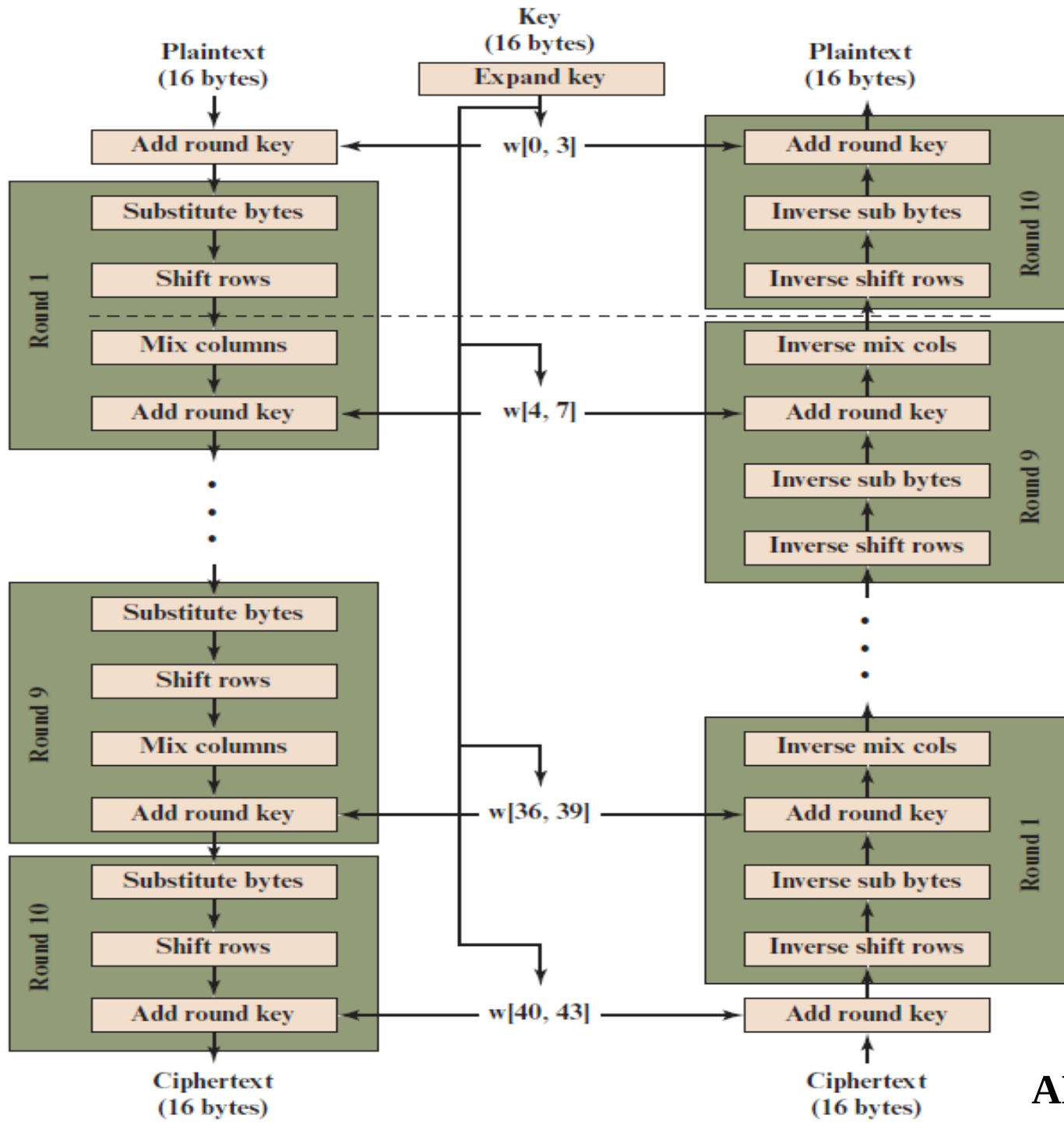
- AES is a non-Feistel cipher that encrypts and decrypts a data block of 128 bits.
- It uses 10, 12, or 14 rounds.
- The key size, which can be 128, 192, or 256 bits, depends on the number of rounds.

Each version uses a different cipher key size (128, 192, or 256), but the round keys are always 128 bits.



No. of rounds	Key Length (bytes)
10	16
12	24
14	32

AES Encryption Process

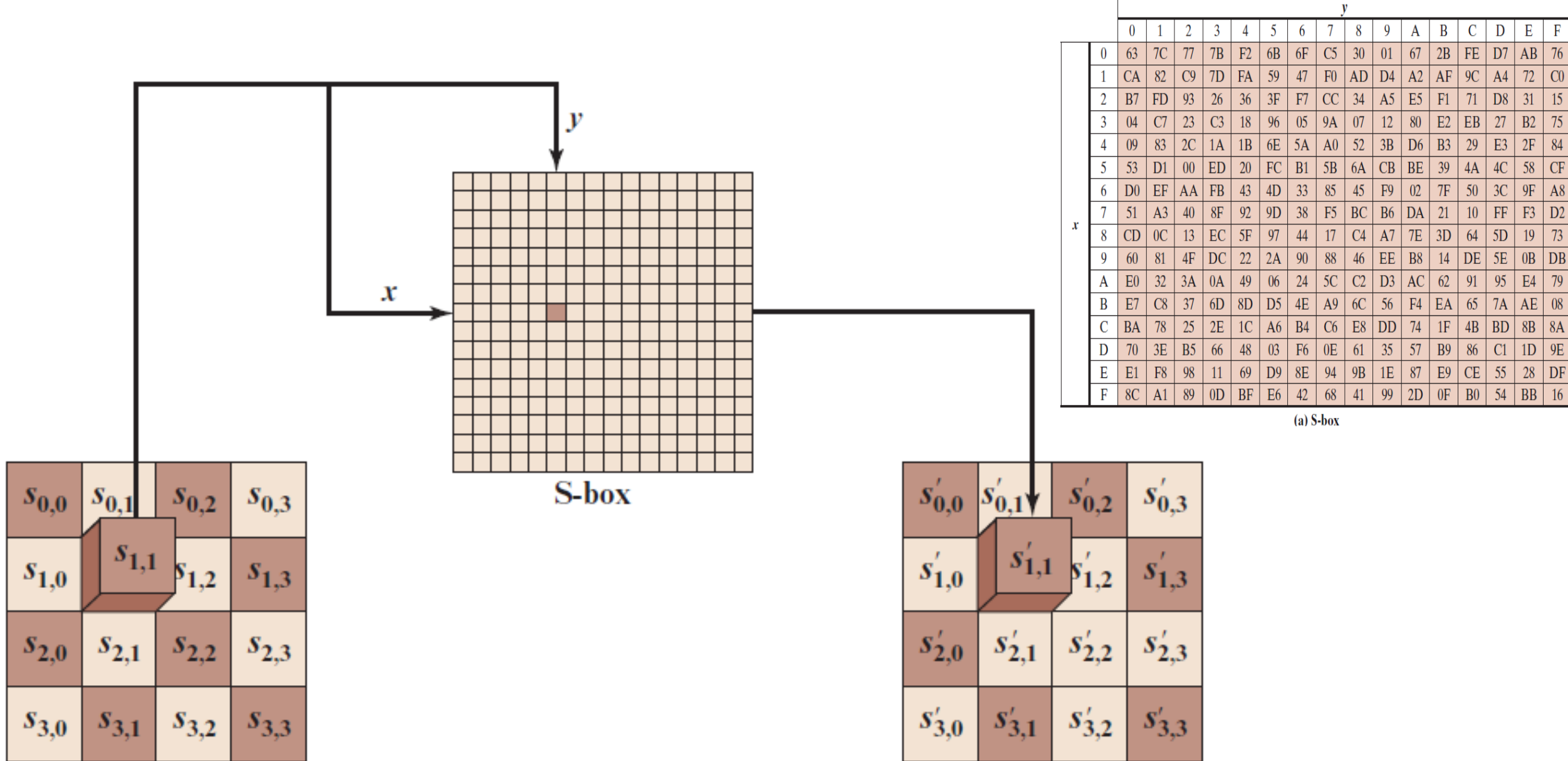


(a) Encryption

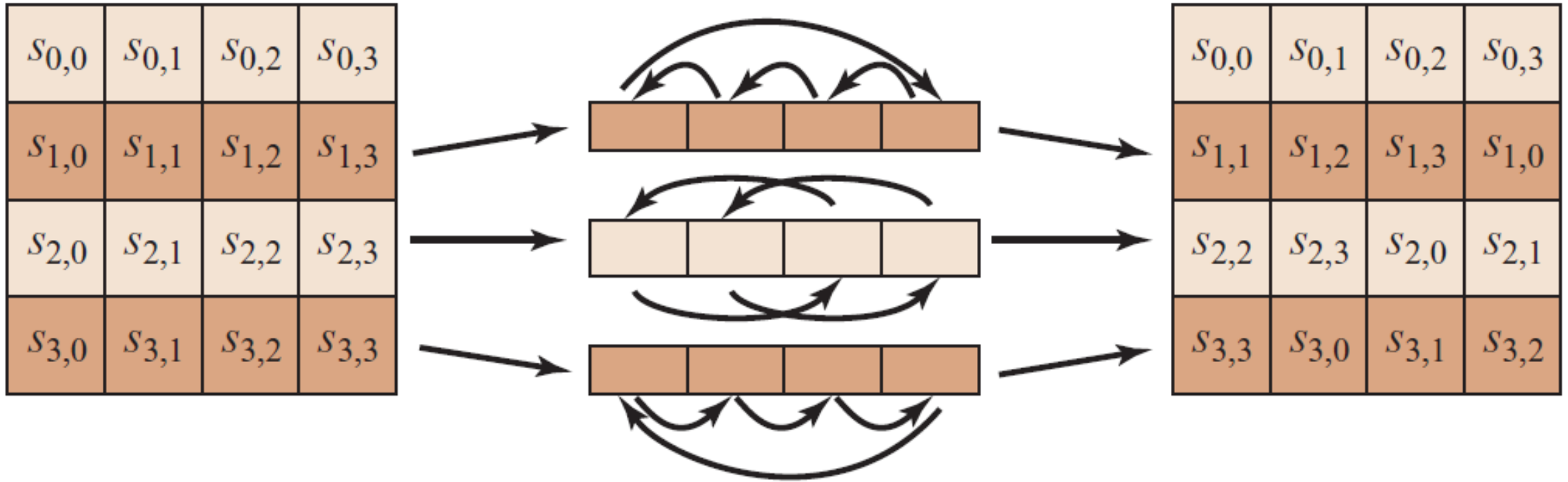
(b) Decryption

AES Transformation functions

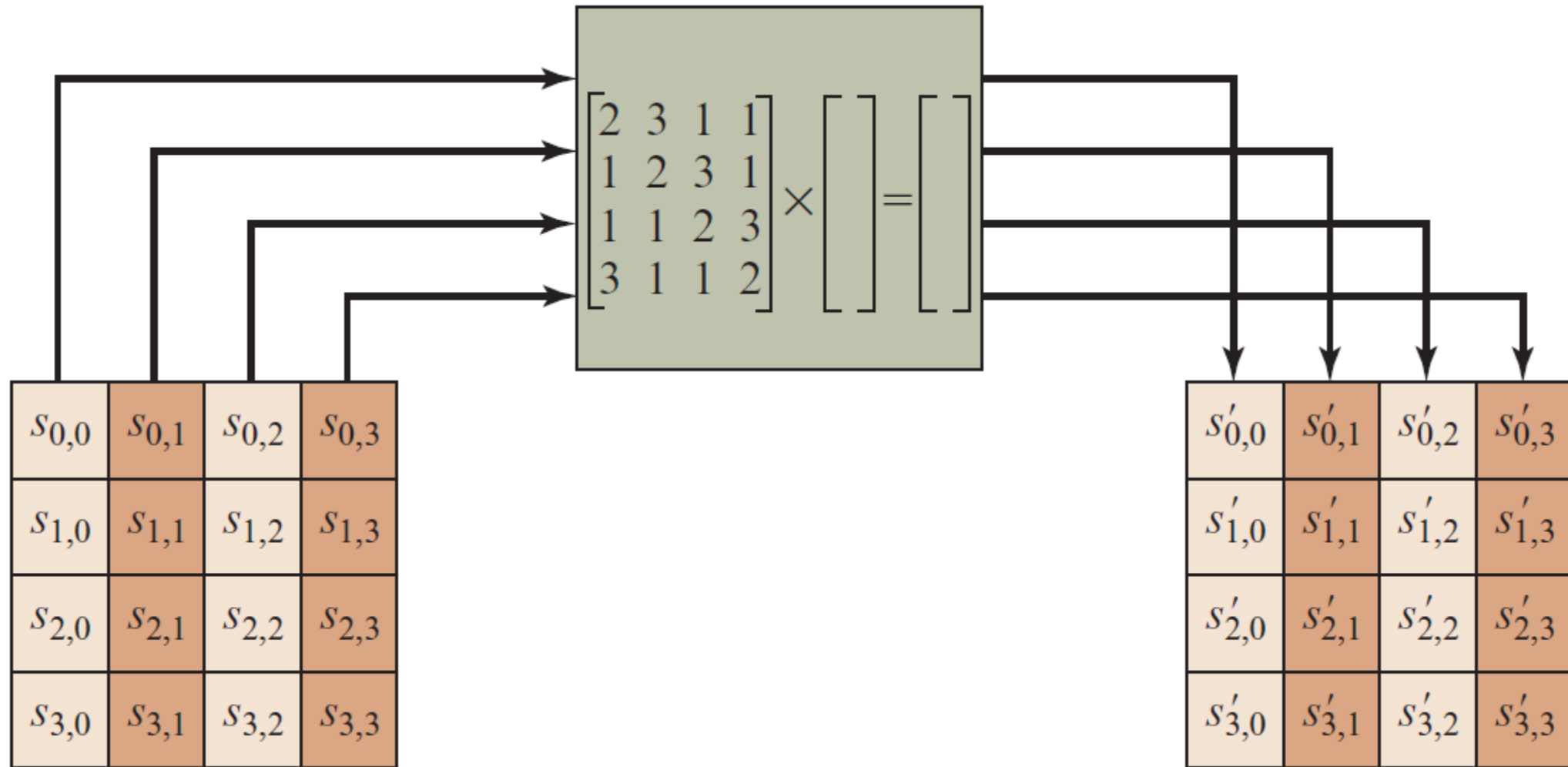
1. Byte substitution (1 S-box used on every byte)
2. Shift rows (permute bytes between groups/columns)
3. Mix columns (subs using matrix multiply of groups)
4. Add round key (XOR state with key material)



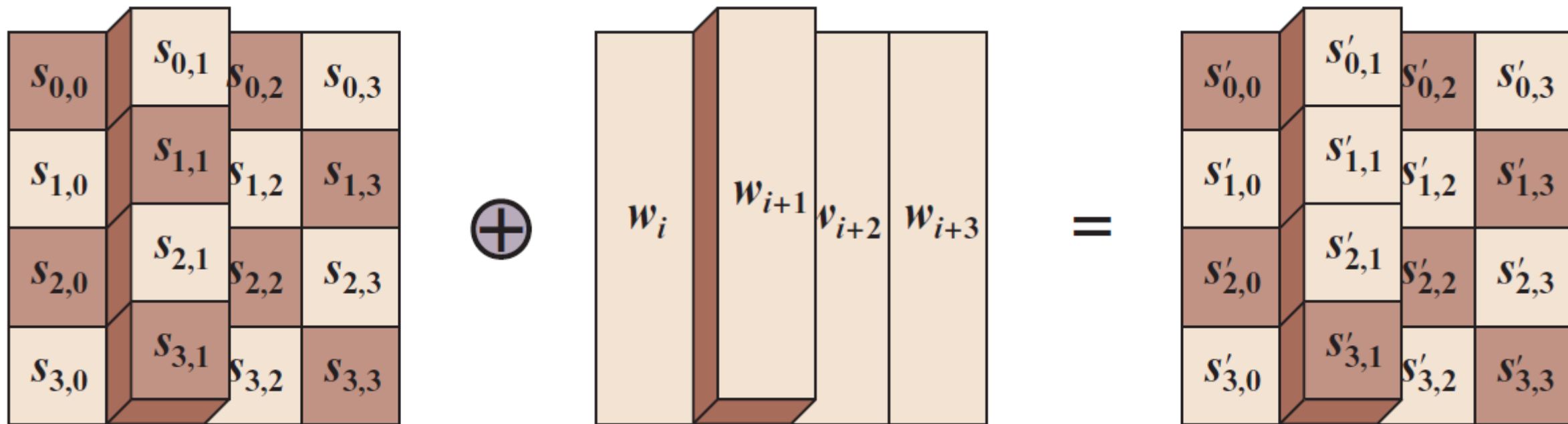
1. Substitute byte transformation



2. Shift row transformation

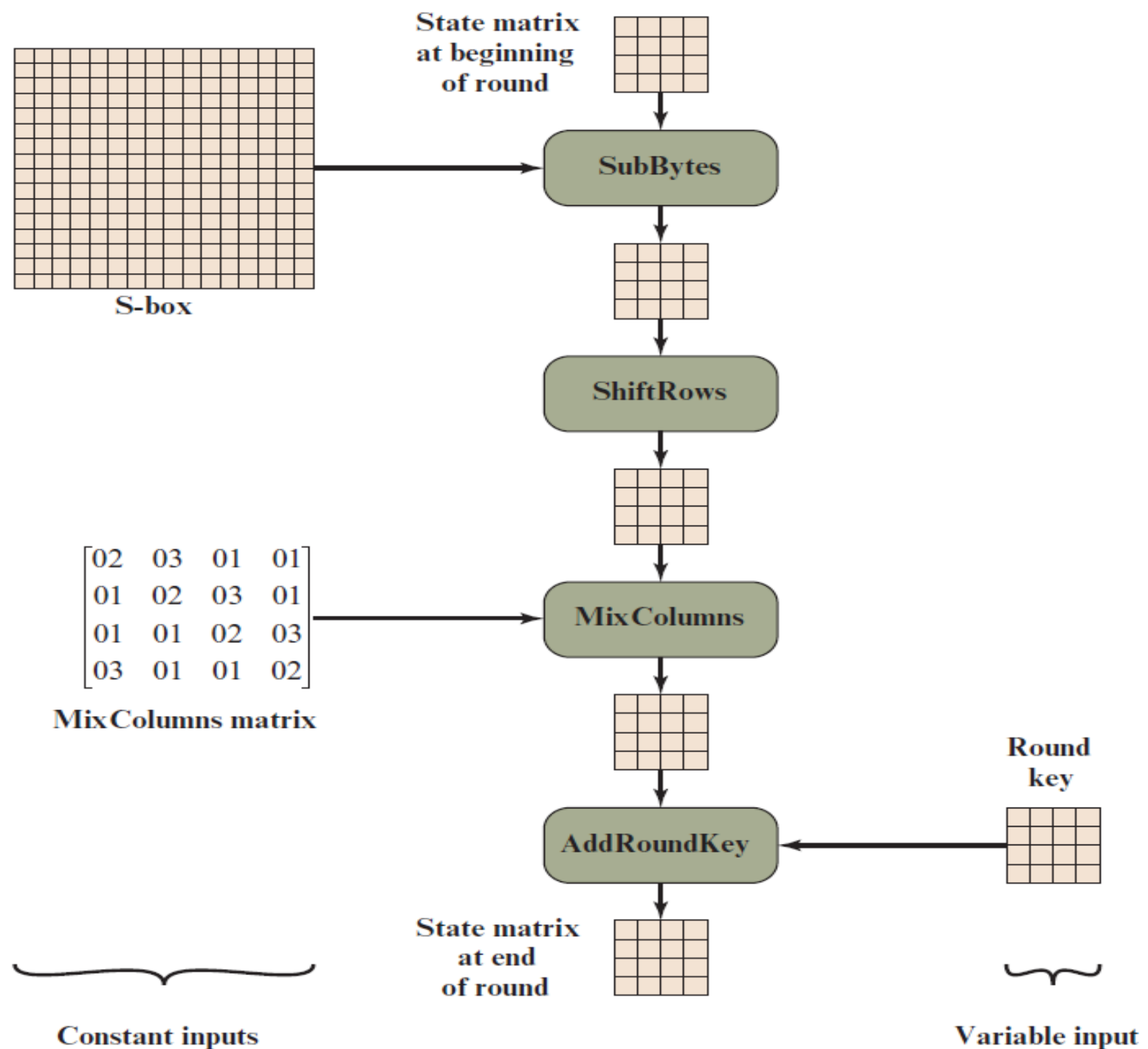


3. Mix column transformation

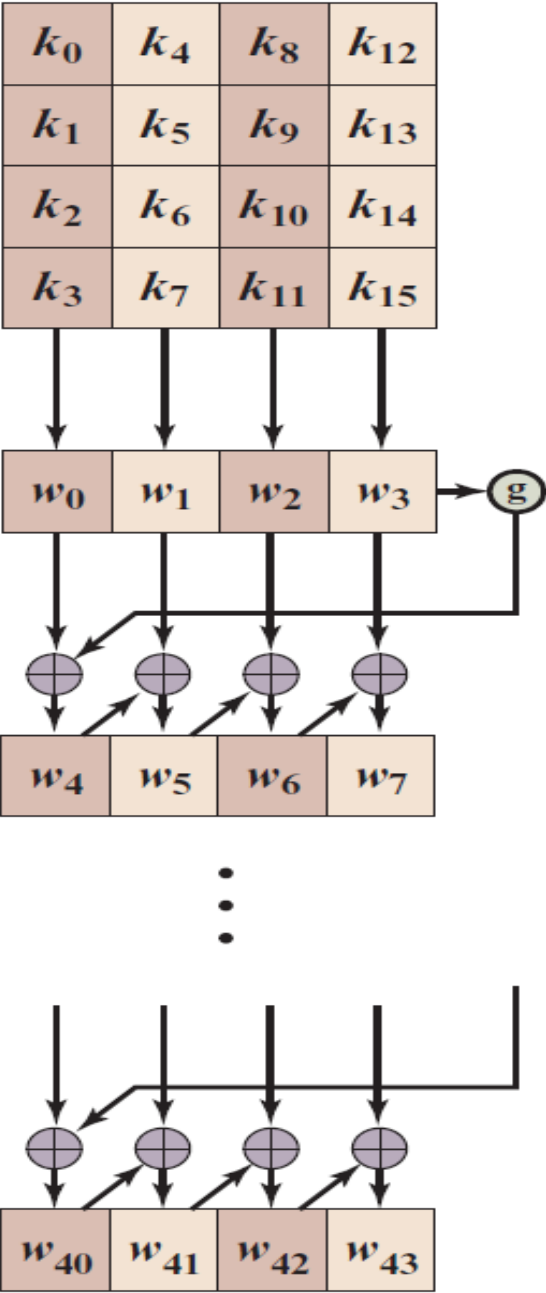


4. Add round key transformation

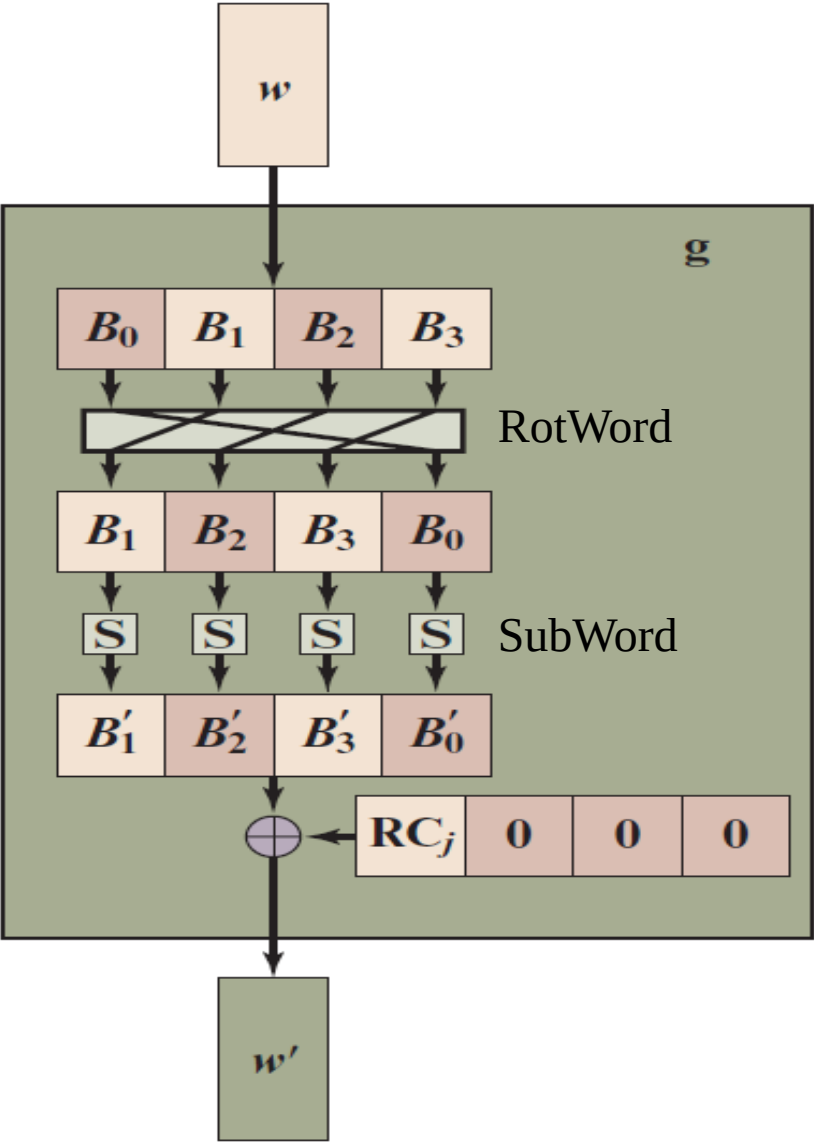
Inputs for Single AES Round



AES Key Expansion



(a) Overall algorithm



(b) Function g

		y															
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
x	0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
	1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
	2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
	3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
	4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
	5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
	6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
	7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
	8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
	9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
	A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
	B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
	C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
	D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
	E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
	F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

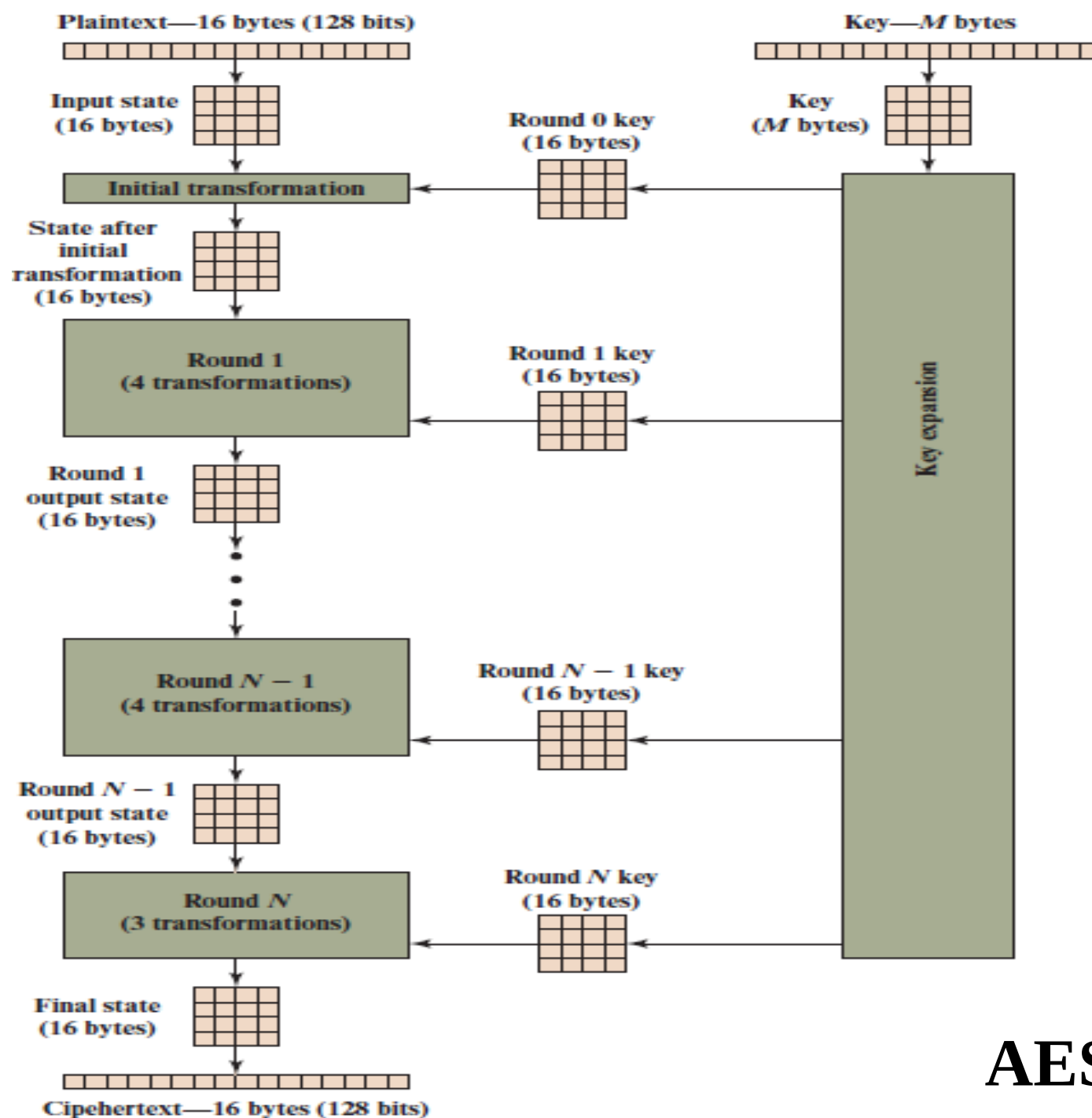
(a) S-box

		y															
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
x	0	52	09	6A	D5	30	36	A5	38	BF	40	A3	9E	81	F3	D7	FB
	1	7C	E3	39	82	9B	2F	FF	87	34	8E	43	44	C4	DE	E9	CB
	2	54	7B	94	32	A6	C2	23	3D	EE	4C	95	0B	42	FA	C3	4E
	3	08	2E	A1	66	28	D9	24	B2	76	5B	A2	49	6D	8B	D1	25
	4	72	F8	F6	64	86	68	98	16	D4	A4	5C	CC	5D	65	B6	92
	5	6C	70	48	50	FD	ED	B9	DA	5E	15	46	57	A7	8D	9D	84
	6	90	D8	AB	00	8C	BC	D3	0A	F7	E4	58	05	B8	B3	45	06
	7	D0	2C	1E	8F	CA	3F	0F	02	C1	AF	BD	03	01	13	8A	6B
	8	3A	91	11	41	4F	67	DC	EA	97	F2	CF	CE	F0	B4	E6	73
	9	96	AC	74	22	E7	AD	35	85	E2	F9	37	E8	1C	75	DF	6E
	A	47	F1	1A	71	1D	29	C5	89	6F	B7	62	0E	AA	18	BE	1B
	B	FC	56	3E	4B	C6	D2	79	20	9A	DB	C0	FE	78	CD	5A	F4
	C	1F	DD	A8	33	88	07	C7	31	B1	12	10	59	27	80	EC	5F
	D	60	51	7F	A9	19	B5	4A	0D	2D	E5	7A	9F	93	C9	9C	EF
	E	A0	E0	3B	4D	AE	2A	F5	B0	C8	EB	BB	3C	83	53	99	61
	F	17	2B	04	7E	BA	77	D6	26	E1	69	14	63	55	21	0C	7D

(b) Inverse S-box

Round Constants in AES-128

Round Number	R Constant (Rcon)
1	(01000000)16
2	(02000000)16
3	(04000000)16
4	(08000000)16
5	(10000000)16
6	(20000000)16
7	(40000000)16
8	(80000000)16
9	(1B000000)16
10	(36000000)16



No. of rounds	Key Length (bytes)
10	16
12	24
14	32

AES Encryption Process

Tools

- Openssl
- <https://legacy.cryptool.org/en/cto/aes-step-by-step>

Advanced Encryption Standard (AES)

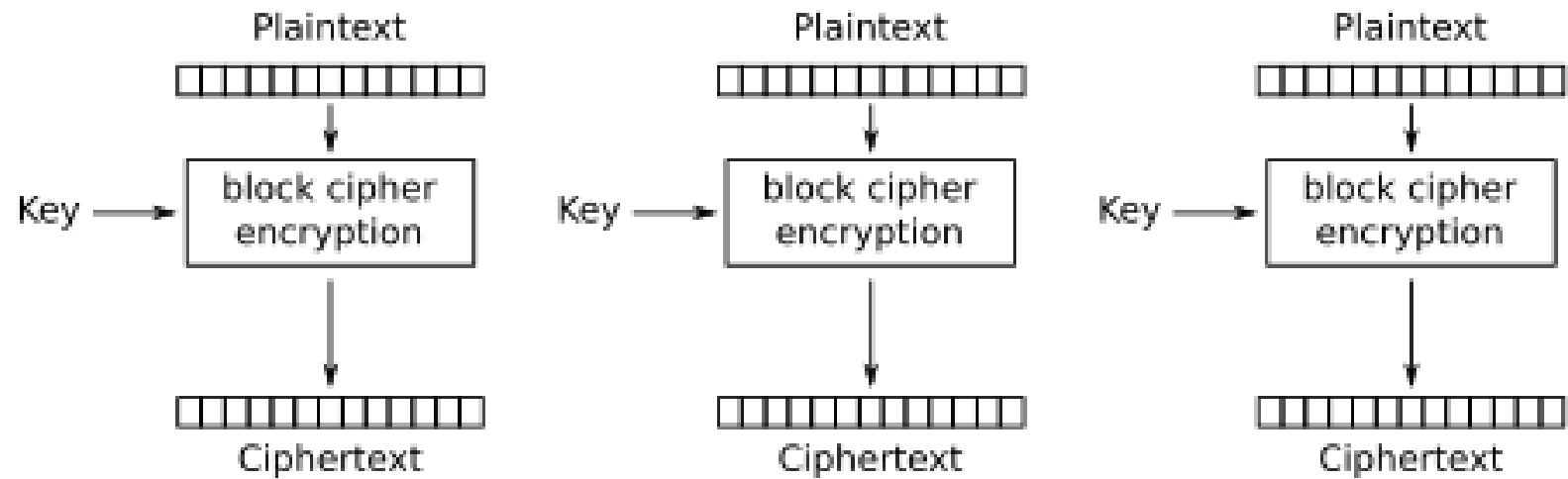
- AES is a highly trusted **encryption algorithm** used to secure data by converting it into an unreadable format without the proper key.
- Developed by the National Institute of Standards and Technology (NIST), **AES encryption** uses various **key lengths** (128, 192, or 256 bits) to provide strong protection against unauthorized access.
- AES is a **Block Cipher**.
- The **key size** can be **128/192/256 bits**.
- Encrypts data in **blocks of 128 bits** each.
- It takes 128 bits as input and outputs 128 bits of encrypted cipher text.
- AES relies on the **substitution-permutation network principle**, which is performed using a series of linked operations that involve replacing and shuffling the input data.

Block cipher mode of operation

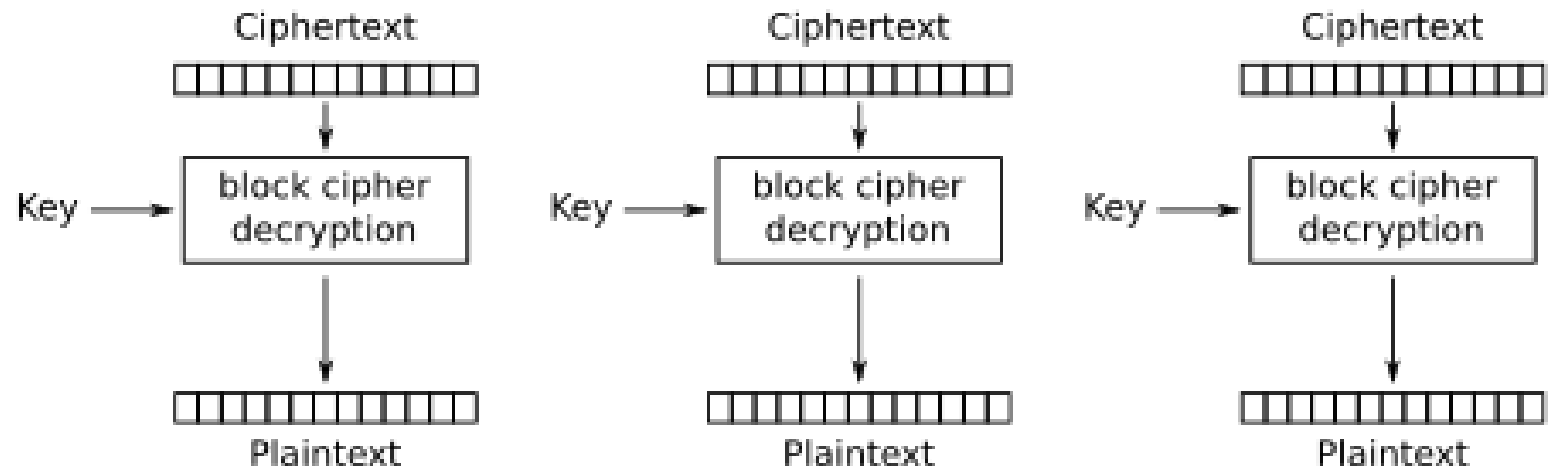
- AES performs operations on bytes of data rather than in bits. Since the block size is 128 bits, the cipher processes 128 bits (or 16 bytes) of the input data at a time.
- The number of rounds depends on the key length as follows :
 - 128-bit key – 10 rounds
 - 192-bit key – 12 rounds
 - 256-bit key – 14 rounds

AES modes of operation

- The **Electronic Code Book (ECB)** mode is one of the easiest and most effective algorithms to use as a **simple replacement** technique.
- The input plaintext is **divided into blocks and encrypted separately** with the key. This enables the decryption of each encrypted block independently.
- **Encrypting the same block twice returns the same ciphertext twice.**



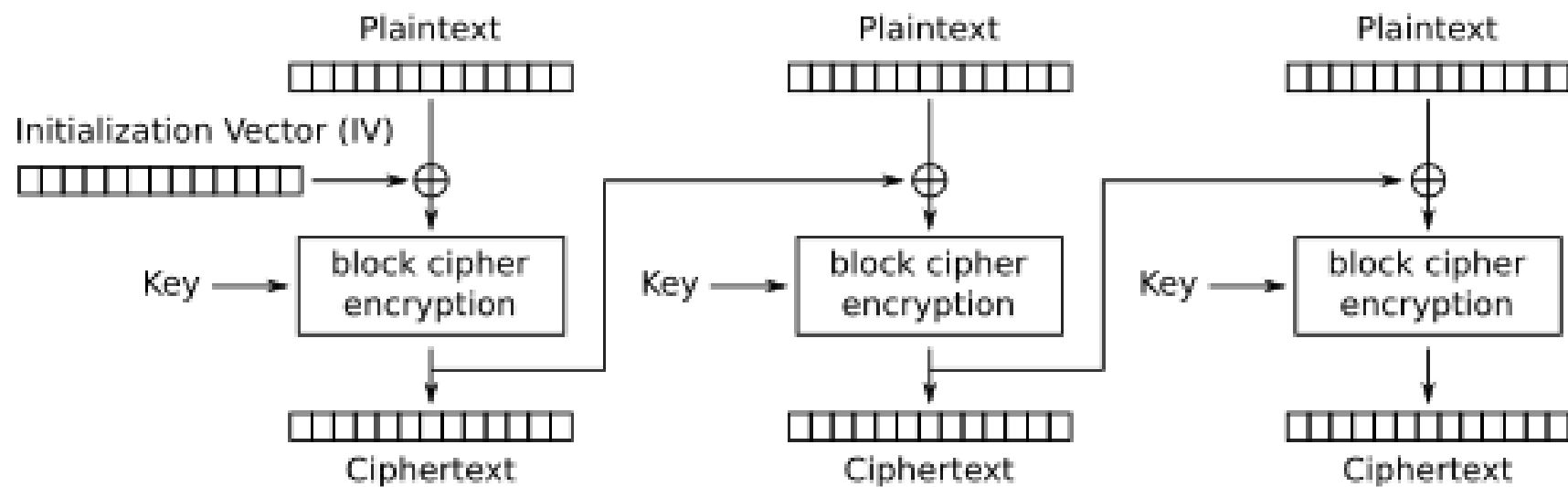
Electronic Codebook (ECB) mode encryption



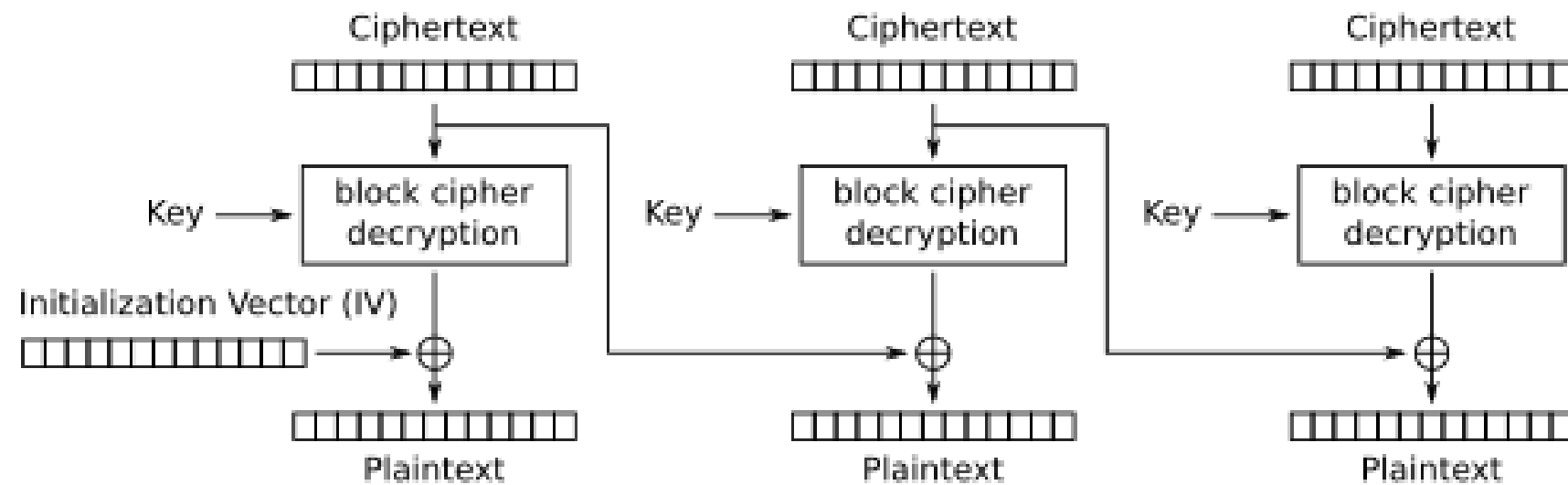
Electronic Codebook (ECB) mode decryption

AES modes of operation

- **CBC (cipher-block chaining)** in cryptography is an AES block cipher mode that improves the ECB mode for **minimizing patterns in plaintext**.
- CBC mode does this by XOR'ing the first plaintext block (B1) with an **initialization vector** before encrypting it.
- CBC also uses **block chaining**, in which each following plaintext block is XOR-ed with the ciphertext of the preceding block.



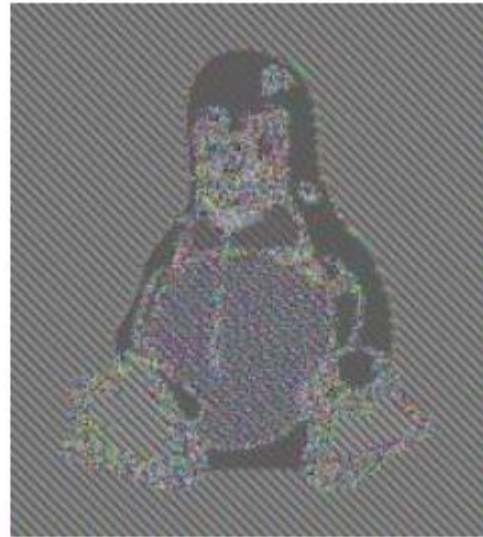
Cipher Block Chaining (CBC) mode encryption



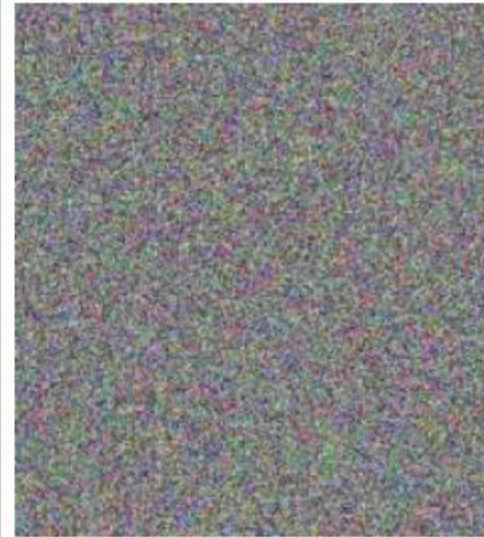
Cipher Block Chaining (CBC) mode decryption



UNENCRYPTED DATA



ECB MODE - ENCRYPTED DATA



CBC MODE - ENCRYPTED DATA

Public-Key Cryptography

Terminology Related to Asymmetric Encryption

Asymmetric Keys

Two related keys, a public key and a private key, that are used to perform complementary operations, such as encryption and decryption or signature generation and signature verification.

Public Key Certificate

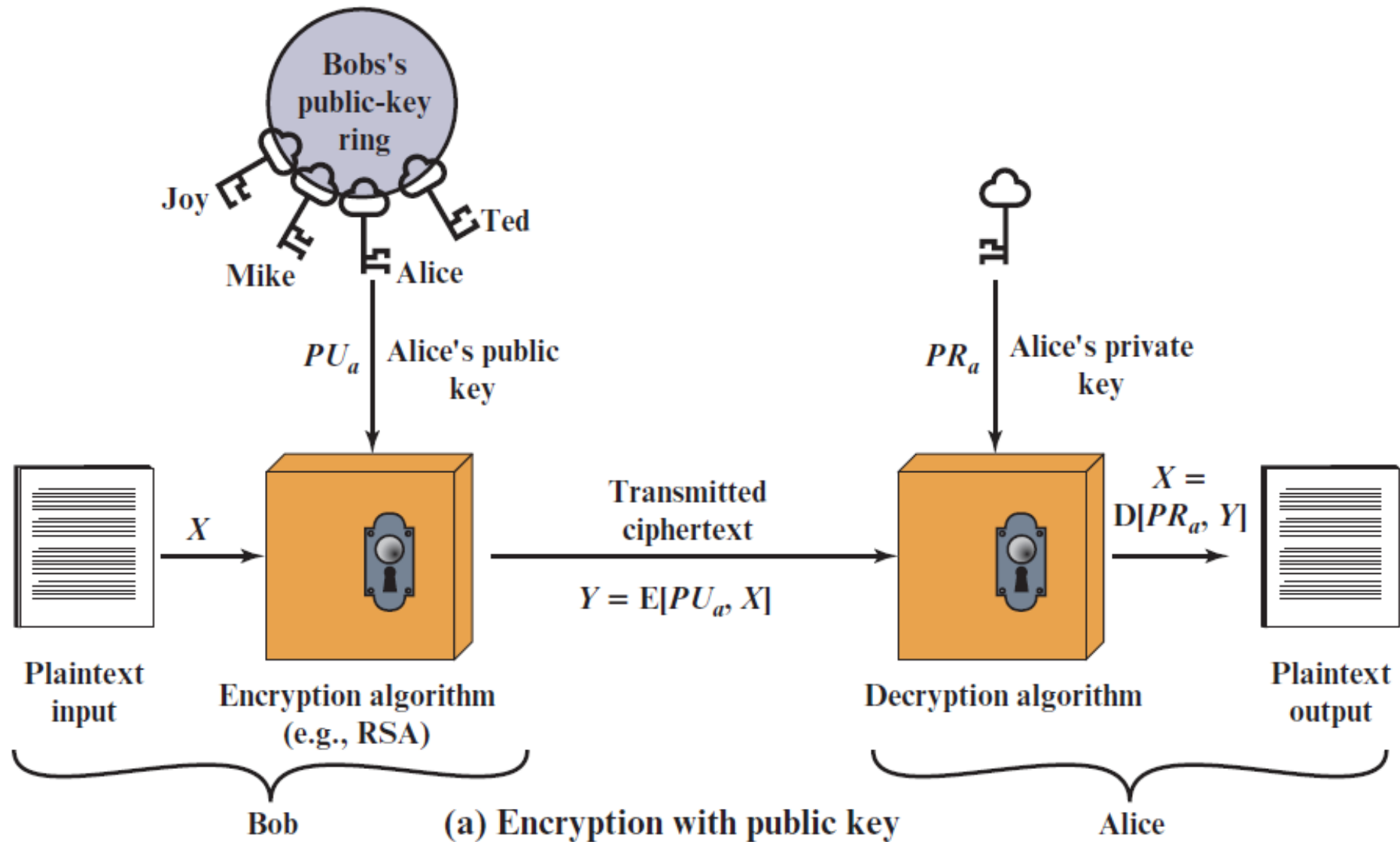
A digital document issued and digitally signed by the private key of a Certification Authority that binds the name of a subscriber to a public key. The certificate indicates that the subscriber identified in the certificate has **sole control and access** to the corresponding private key.

Public Key (Asymmetric) Cryptographic Algorithm

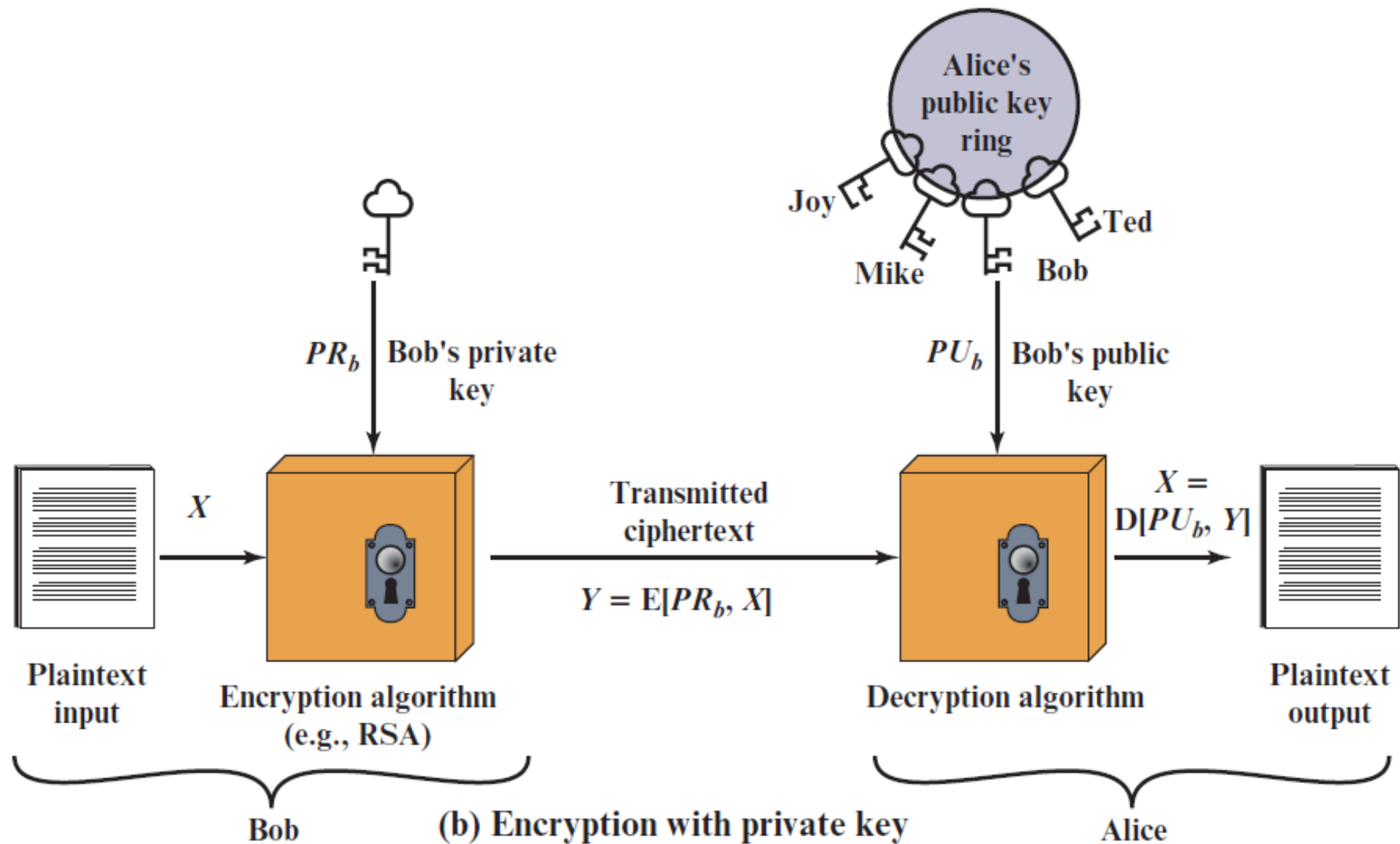
A cryptographic algorithm that uses two related keys, a public key and a private key. The two keys have the property that deriving the private key from the public key is computationally infeasible.

Public Key Infrastructure (PKI)

A set of **policies, processes, server platforms, software and workstations** used for the purpose of **administering certificates and public-private key pairs**, including the ability to **issue, maintain, and revoke** public key certificates.



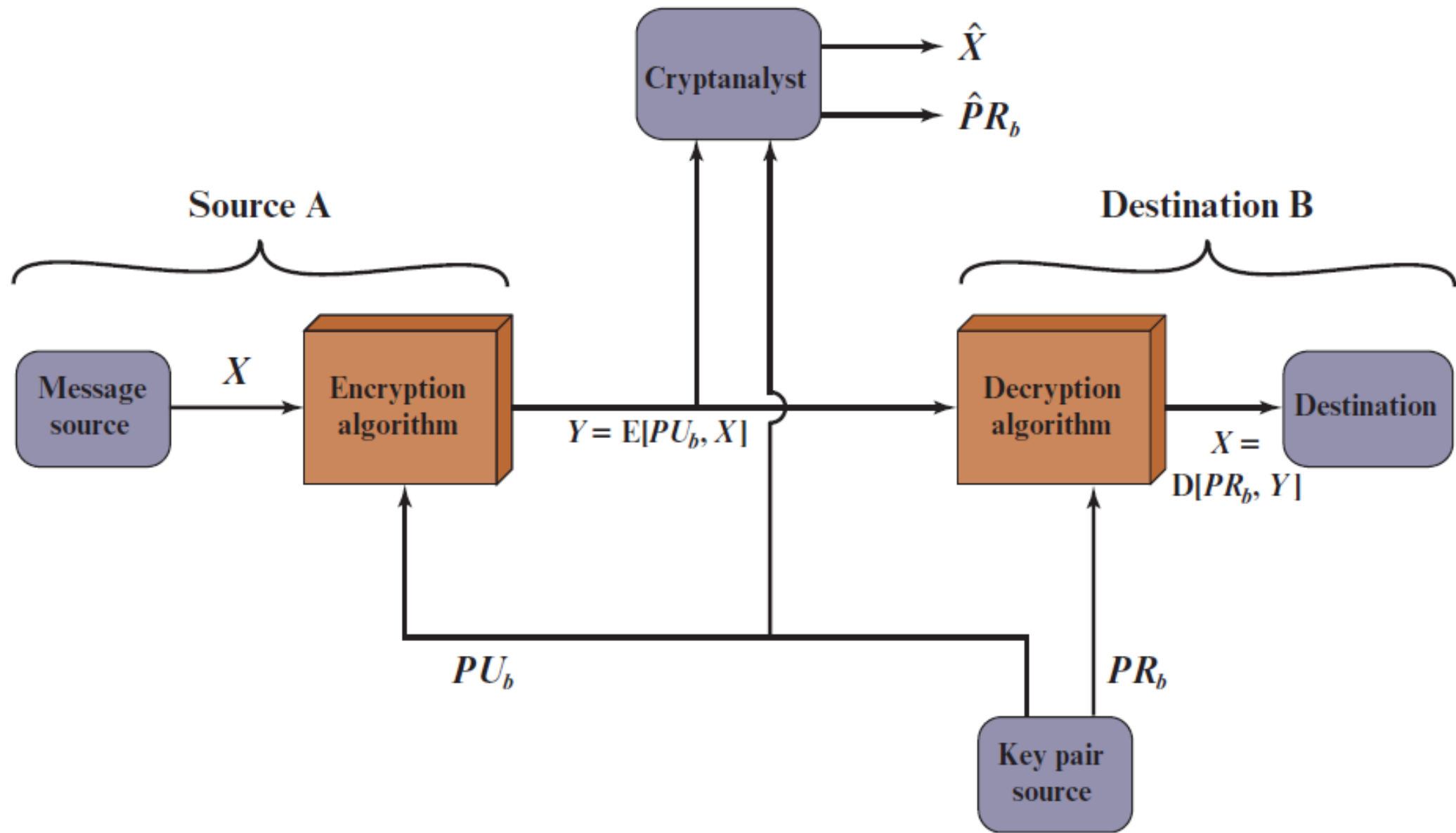
Public-Key Cryptography



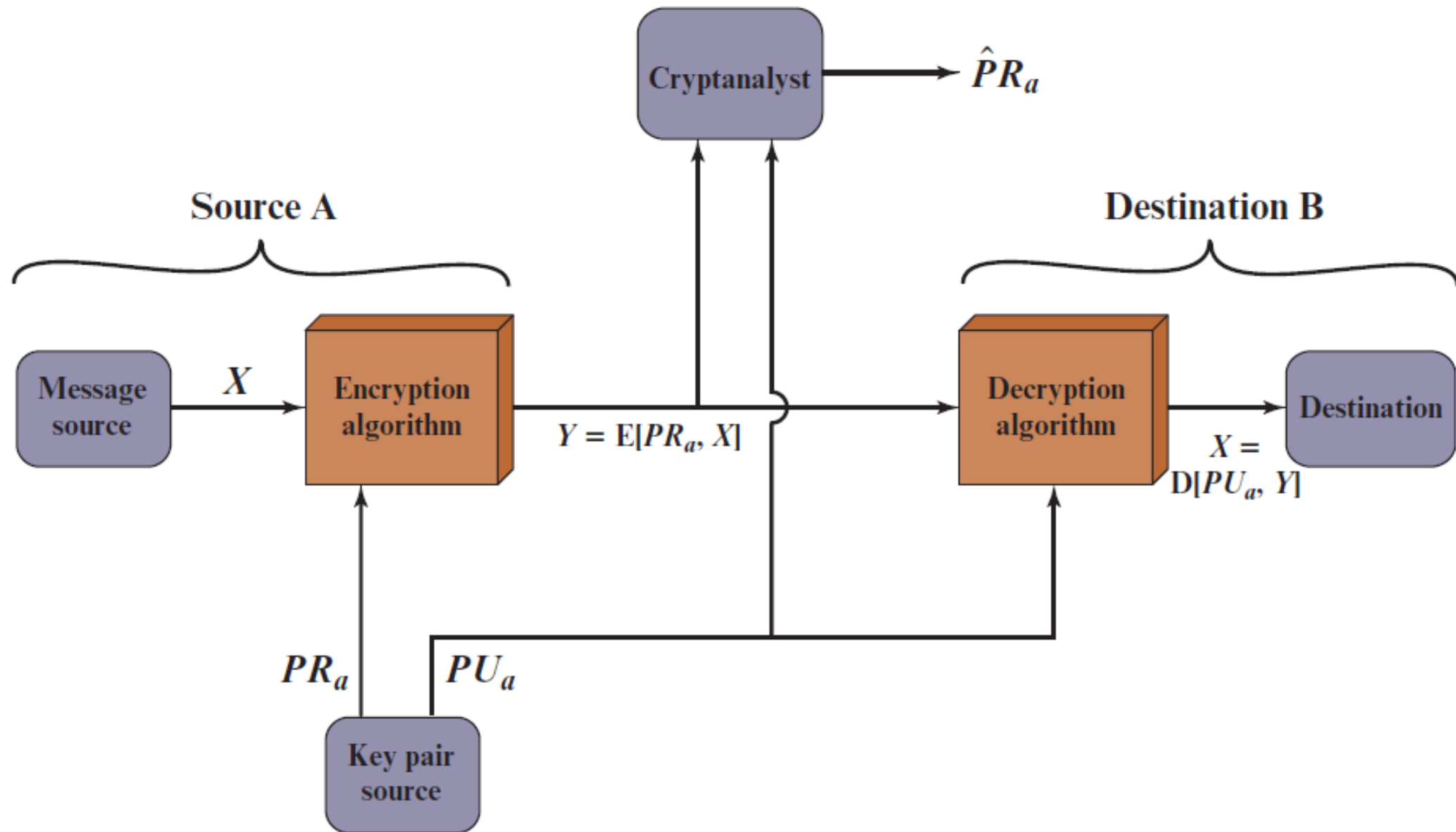
Public-Key Cryptography

Conventional and Public-Key Encryption

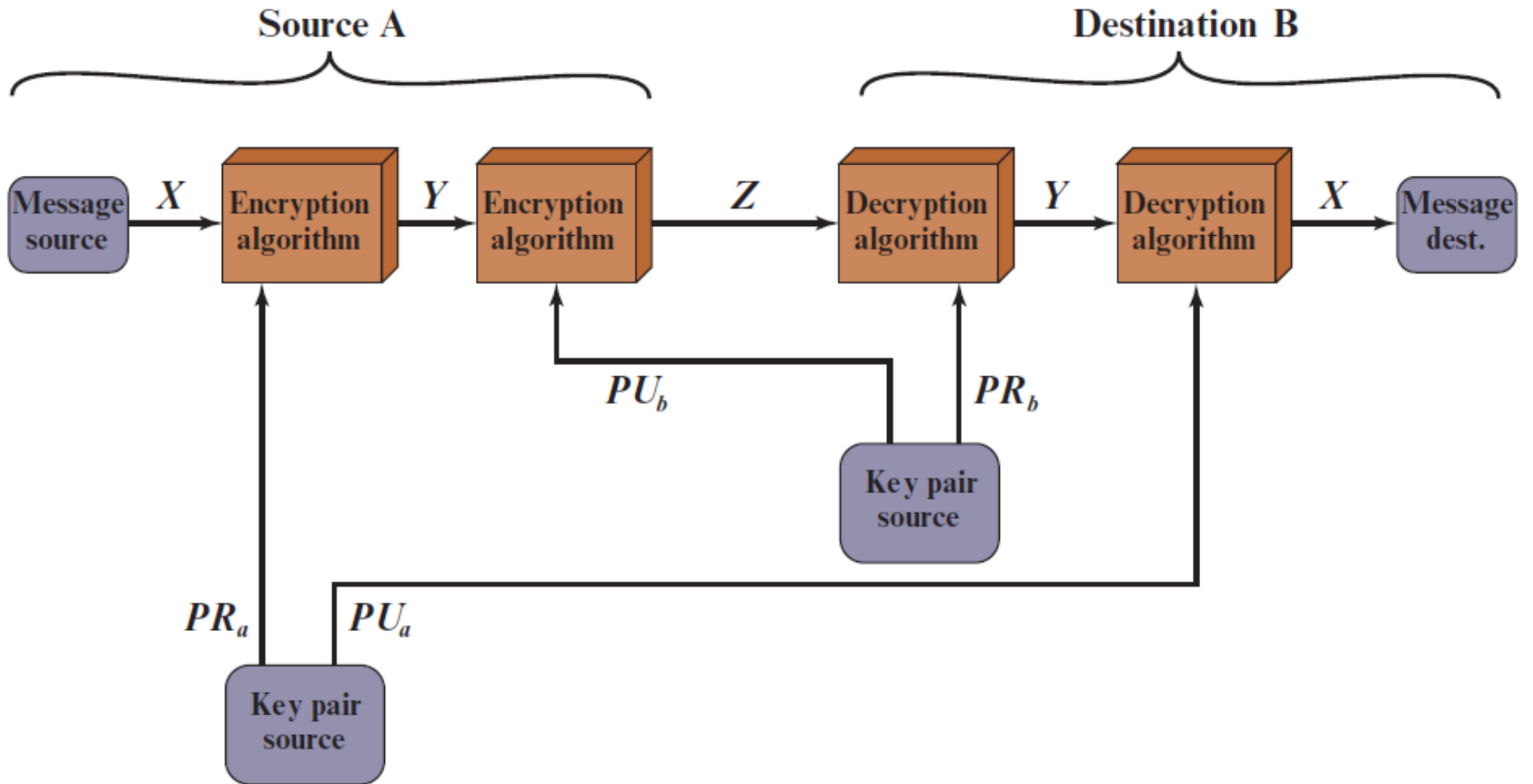
Conventional Encryption	Public-Key Encryption
<i>Needed to Work:</i> 1. The same algorithm with the same key is used for encryption and decryption. 2. The sender and receiver must share the algorithm and the key. <i>Needed for Security:</i> 1. The key must be kept secret. 2. It must be impossible or at least impractical to decipher a message if the key is kept secret. 3. Knowledge of the algorithm plus samples of ciphertext must be insufficient to determine the key.	<i>Needed to Work:</i> 1. One algorithm is used for encryption and a related algorithm for decryption with a pair of keys, one for encryption and one for decryption. 2. The sender and receiver must each have one of the matched pair of keys (not the same one). <i>Needed for Security:</i> 1. One of the two keys must be kept secret. 2. It must be impossible or at least impractical to decipher a message if one of the keys is kept secret. 3. Knowledge of the algorithm plus one of the keys plus samples of ciphertext must be insufficient to determine the other key.



Public-Key Cryptosystem: Confidentiality



Public-Key Cryptosystem: Authentication



Public-Key Cryptosystem: Authentication and Secrecy

Applications for Public-Key Cryptosystems

Algorithm	Encryption/Decryption	Digital Signature	Key Exchange
RSA	Yes	Yes	Yes
Elliptic Curve	Yes	Yes	Yes
Diffie–Hellman	No	No	Yes
DSS	No	Yes	No

Public-Key Cryptanalysis

- As with symmetric encryption, a public-key encryption scheme is vulnerable to a **brute-force attack**.
- The countermeasure is the same: **Use large keys**.
- However, there is a tradeoff to be considered.
- Public-key systems depend on the use of some sort of **invertible mathematical function**.
- The **larger the key**, the more computationally **expensive** the cryptographic operations become.
- Thus, the key size must be **large enough** to make **brute-force attack impractical** but **small enough** for **practical encryption and decryption**.

THE RSA ALGORITHM

- RSA was developed in 1977 by Ron Rivest, Adi Shamir, and Len Adleman at MIT and first published in 1978.
- The plaintext and ciphertext are integers between 0 and $n - 1$ for some n .
- A typical size for n is 1024 bits, or 309 decimal digits. That is, n is less than 2^{1024} .

RSA Encryption Algorithm

- **Rivest, Shamir, and Adleman (RSA)**
- RSA encryption algorithm is a type of public-key encryption algorithm.
- Asymmetric algorithms are those algorithms in which sender and receiver use different keys for encryption and decryption. Each sender is assigned a pair of keys:
- **Public key**
- **Private key**
- The **Public key** is used for encryption, and the **Private Key** is used for decryption.
- **The two keys are linked, but the private key cannot be derived from the public key.**
- The public key is well known, but the private key is secret and it is known only to the user who owns the key. It means that everybody can send a message to the user using user's public key. But only the user can decrypt the message using his private key.

RSA Algorithm

- RSA makes use of an **expression with exponentials**.
- Plaintext is **encrypted in blocks**, with each block having a binary value less than some number n .
- That is, the block size must be less than or equal to $\log_2(n) + 1$; in practice, the block size is i bits, where $2^i < n \leq 2^{i+1}$.
- Encryption and decryption are of the following form, for some plaintext block M and ciphertext block C .

$$C = M^e \bmod n$$

$$M = C^d \bmod n = (M^e)^d \bmod n = M^{ed} \bmod n$$

RSA Algorithm

- Both sender and receiver must know the value of n . The sender knows the value of e , and only the receiver knows the value of d .
- Thus, this is a public-key encryption algorithm with a public key of $PU = \{e, n\}$ and a private key of $PR = \{d, n\}$.
- For this algorithm to be satisfactory for public-key encryption, the following requirements must be met.
 1. It is possible to find values of e, d , and n such that $M^{ed} \bmod n = M$ for all $M < n$.
 2. It is relatively easy to calculate $M^e \bmod n$ and $C^d \bmod n$ for all values of $M < n$.
 3. It is infeasible to determine d given e and n .

The RSA Algorithm

Key Generation by Alice

Select p, q

p and q both prime, $p \neq q$

Calculate $n = p \times q$

Calculate $\phi(n) = (p - 1)(q - 1)$

Select integer e

$\gcd(\phi(n), e) = 1; 1 < e < \phi(n)$

Calculate d

$d \equiv e^{-1} \pmod{\phi(n)}$

Public key

$PU = \{e, n\}$

Private key

$PR = \{d, n\}$

Encryption by Bob with Alice's Public Key

Plaintext:

$M < n$

Ciphertext:

$C = M^e \pmod n$

Decryption by Alice with Alice's Private Key

Ciphertext:

C

Plaintext:

$M = C^d \pmod n$

Modular Operations

- **Modular Addition:** $(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$
- **Modular Multiplication:** $(a \times b) \bmod m = ((a \bmod m) \times (b \bmod m)) \bmod m$
- **Modular Division**

$(a / b) \bmod m$ is not equal to $((a \bmod m) / (b \bmod m)) \bmod m$.

$(a / b) \bmod m = (a \times (\text{inverse of } b \text{ if exists})) \bmod m$

- The modular inverse of $a \bmod m$ exists only if a and m are relatively prime i.e. $\gcd(a, m) = 1$. Hence, for finding the inverse of a under modulo m , if $(a \times b) \bmod m = 1$ then b is the Modular Inverse of a .

- **Modular Exponentiation**

$$x^n = \begin{cases} 1 & n = 0 \\ x^{n/2} \cdot x^{n/2} & n \text{ is even} \\ x^{n-1} \cdot x & n \text{ is odd} \end{cases}$$

$$\mathbf{A}^2 \bmod C = (\mathbf{A} * \mathbf{A}) \bmod C = ((\mathbf{A} \bmod C) * (\mathbf{A} \bmod C)) \bmod C$$

RSA Algorithm: Example

1. Select two prime numbers, $p = 17$ and $q = 11$.
2. Calculate $n = pq = 17 \times 11 = 187$.
3. Calculate $\phi(n) = (p - 1)(q - 1) = 16 \times 10 = 160$.
4. Select e such that e is relatively prime to $\phi(n) = 160$ and less than $\phi(n)$; we choose $e = 7$.
5. Determine d such that $de \equiv 1 \pmod{160}$ and $d < 160$. The correct value is $d = 23$, because $23 \times 7 = 161 = (1 \times 160) + 1$; d can be calculated using

$$C = M^e \pmod{n}$$

$$M = C^d \pmod{n}$$

public key $PU = \{7, 187\}$ and private key $PR = \{23, 187\}$.

$M = 88$

$$88^7 \pmod{187} = [(88^4 \pmod{187}) \times (88^2 \pmod{187}) \times (88^1 \pmod{187})] \pmod{187}$$

$$88^1 \pmod{187} = 88$$

$$88^2 \pmod{187} = 7744 \pmod{187} = 77$$

$$88^4 \pmod{187} = 59,969,536 \pmod{187} = 132$$

$$88^7 \pmod{187} = (88 \times 77 \times 132) \pmod{187} = 894,432 \pmod{187} = 11$$

$$11^{23} \pmod{187} = [(11^1 \pmod{187}) \times (11^2 \pmod{187}) \times (11^4 \pmod{187}) \times (11^8 \pmod{187}) \times (11^8 \pmod{187})] \pmod{187}$$

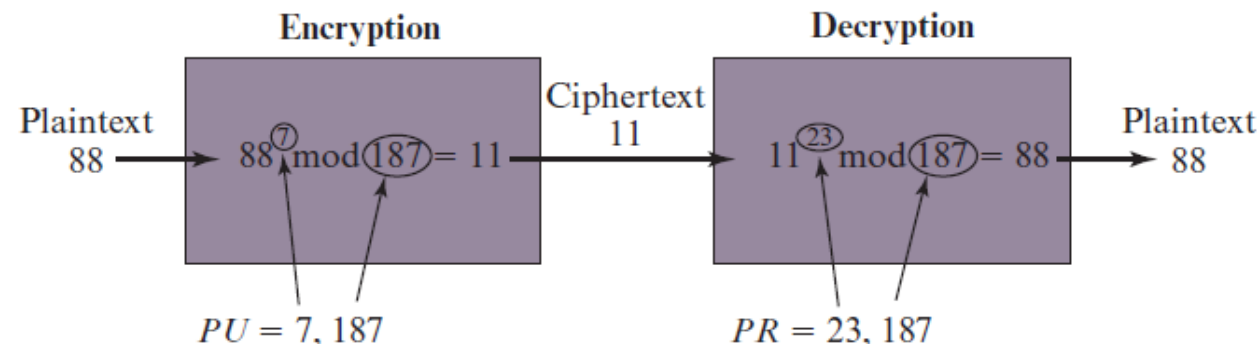
$$11^1 \pmod{187} = 11$$

$$11^2 \pmod{187} = 121$$

$$11^4 \pmod{187} = 14,641 \pmod{187} = 55$$

$$11^8 \pmod{187} = 214,358,881 \pmod{187} = 33$$

$$11^{23} \pmod{187} = (11 \times 121 \times 55 \times 33 \times 33) \pmod{187} = 79,720,245 \pmod{187} = 88$$



RSA Example

- Solve the RSA algorithm by performing encryption and decryption using the following: $p=3$, $q=11$, $M=2$.
- To find: n , e , d

RSA Example

- Choose $p = 3$ and $q = 11$
- Compute $n = p * q = 3 * 11 = 33$
- Compute $\phi(n) = (p - 1) * (q - 1) = 2 * 10 = 20$
- Choose e such that $1 < e < \phi(n)$ and e and $\phi(n)$ are coprime. Let $e = 7$
- Compute a value for d such that $(d * e) \% \phi(n) = 1$.
- One solution is $d = 3$ $[(3 * 7) \% 20 = 1]$
- Public key is $(e, n) \Rightarrow (7, 33)$
- Private key is $(d, n) \Rightarrow (3, 33)$
- The encryption of $m = 2$ is $c = 2^7 \% 33 = 29$
- The decryption of $c = 29$ is $m = 29^3 \% 33 = 2$

Generate public and private keys:

- Select two large prime numbers, p and q . Multiply these numbers to find $n = p \times q$, where n is called the modulus for encryption and decryption.
- Choose a number e less than n , such that n is relatively prime to $(p - 1) \times (q - 1)$. It means that e and $(p - 1) \times (q - 1)$ have no common factor except 1. Choose "e" such that $1 < e < \phi(n)$, e is prime to $\phi(n)$,
 $\gcd(e, \phi(n)) = 1$
- If $n = p \times q$, then the public key is $\langle e, n \rangle$. A plaintext message m is encrypted using public key $\langle e, n \rangle$. To find ciphertext from the plain text following formula is used to get ciphertext C .

$$C = m^e \bmod n$$

Here, m must be less than n . A larger message ($>n$) is treated as a concatenation of messages, each of which is encrypted separately.

- To determine the private key, we use the following formula to calculate the d such that:
 $d \cdot e \bmod \{(p - 1) \times (q - 1)\} = 1$ Or $d \cdot e \bmod \phi(n) = 1$
- The private key is $\langle d, n \rangle$. A ciphertext message c is decrypted using private key $\langle d, n \rangle$. To calculate plain text m from the ciphertext c following formula is used to get plain text m .

$$m = c^d \bmod n$$

Encrypt plaintext 9 using the RSA public-key encryption algorithm. This example uses prime numbers 7 and 11 to generate the public and private keys.

Step 1: Select two large prime numbers, p , and q .

- $p = 7$
- $q = 11$

Step 2: Multiply these numbers to find $n = p \times q$, where n is called the modulus for encryption and decryption.

- First, we calculate
- $n = p \times q$
- $n = 7 \times 11$
- $n = 77$

Step 3: Choose a number **e** less than **n**, such that **n** is relatively prime to **(p - 1) x (q - 1)**. It means that **e** and **(p - 1) x (q - 1)** have no common factor except 1. Choose "e" such that $1 < e < \phi(n)$, e is prime to $\phi(n)$, $\gcd(e, \phi(n)) = 1$.

- Second, we calculate
- **$\phi(n) = (p - 1) \times (q - 1)$**
- $\phi(n) = (7 - 1) \times (11 - 1)$
- $\phi(n) = 6 \times 10$
- $\phi(n) = 60$
- Let us now choose relative prime e of 60 as 7.
- Thus the public key is $\langle e, n \rangle = (7, 77)$

Step 4: A plaintext message **m** is encrypted using public key $\langle e, n \rangle$. To find ciphertext from the plain text following formula is used to get ciphertext C.

- To find ciphertext from the plain text following formula is used to get ciphertext C.
- **$C = m^e \bmod n$**
- $C = 9^7 \bmod 77$
- $C = 37$

Step 5: The private key is $\langle d, n \rangle$. To determine the private key, we use the following formula d such that:

- $d \cdot e \bmod \{(p - 1) \times (q - 1)\} = 1$

$$d = \frac{1 + (k \cdot \phi(n))}{e}$$

- $7d \bmod 60 = 1$, which gives $d = 43$

- The private key is $\langle d, n \rangle = (43, 77)$

Step 6: A ciphertext message c is decrypted using private key $\langle d, n \rangle$. To calculate plain text m from the ciphertext c following formula is used to get plain text m .

- $m = c^d \bmod n$

- $m = 37^{43} \bmod 77$

- $m = 9$

- In this example, Plain text = 9 and the ciphertext = 37

DIFFIE–HELLMAN KEY EXCHANGE

- The purpose of the algorithm is to **enable two users to securely exchange a key** that can then be used for subsequent symmetric encryption of messages.
- The Diffie–Hellman algorithm depends for its **effectiveness** on the **difficulty of computing discrete logarithms**.
- Based on - A **primitive root** of a prime number p is one whose powers modulo p generate all the integers from 1 to $p - 1$.
- That is, if a is a primitive root of the prime number p , then the numbers **$a \bmod p, a^2 \bmod p, \dots, a^{p-1} \bmod p$** are distinct and consist of the integers from 1 through $p - 1$ in some permutation.

The Diffie–Hellman Key Exchange



Alice



Bob

Alice and Bob share a prime number q and an integer α , such that $\alpha < q$ and α is a primitive root of q

Alice generates a private key X_A such that $X_A < q$

Alice calculates a public key $Y_A = \alpha^{X_A} \bmod q$

Alice receives Bob's public key Y_B in plaintext

Alice calculates shared secret key $K = (Y_B)^{X_A} \bmod q$



Alice and Bob share a prime number q and an integer α , such that $\alpha < q$ and α is a primitive root of q

Bob generates a private key X_B such that $X_B < q$

Bob calculates a public key $Y_B = \alpha^{X_B} \bmod q$

Bob receives Alice's public key Y_A in plaintext

Bob calculates shared secret key $K = (Y_A)^{X_B} \bmod q$



$$\begin{aligned} K &= (Y_B)^{X_A} \bmod q \\ &= (\alpha^{X_B} \bmod q)^{X_A} \bmod q \\ &= (\alpha^{X_B})^{X_A} \bmod q \\ &= \alpha^{X_B X_A} \bmod q \\ &= (\alpha^{X_A})^{X_B} \bmod q \\ &= (\alpha^{X_A} \bmod q)^{X_B} \bmod q \\ &= (Y_A)^{X_B} \bmod q \end{aligned}$$

Example : Diffie Hellman

- For this scheme, there are two publicly known numbers: a prime number q and an integer α that is a primitive root of q .

- Let $q=17$, $\alpha < q$, $\alpha=5$

- Let $X_A=4$, $Y_A = 5^4 \bmod 17 = 13$

- Let $X_B = 6$, $Y_B = 5^6 \bmod 17 = 2$

- $K_A = 2^4 \bmod 17 = 16$ $K = (Y_B)^{X_A} \bmod q$

- $K_B = 13^6 \bmod 17 = 16$ $K = (Y_A)^{X_B} \bmod q$

$$5^1 \bmod 17 = 5$$

$$5^2 \bmod 17 = 7$$

$$5^3 \bmod 17 = 6$$

$$5^4 \bmod 17 = 13$$

$$5^5 \bmod 17 = 14$$

$$5^6 \bmod 17 = 2$$

$$5^7 \bmod 17 = 10$$

$$5^8 \bmod 17 = 16$$

$$5^9 \bmod 17 = 12$$

$$5^{10} \bmod 17 = 9$$

$$5^{11} \bmod 17 = 11$$

$$5^{12} \bmod 17 = 4$$

$$5^{13} \bmod 17 = 3$$

$$5^{14} \bmod 17 = 15$$

$$5^{15} \bmod 17 = 7$$

$$5^{16} \bmod 17 = 1$$

Example : Diffie Hellman

- For this scheme, there are two publicly known numbers: a prime number q and an integer α that is a primitive root of q .
- Let $q=7$, $\alpha < q$, $\alpha=5$

Example : Diffie Hellman

- For this scheme, there are two publicly known numbers: a prime number q and an integer α that is a primitive root of q .
 $5^1 \bmod 7 = 5$
- Let $q=7$, $\alpha < q$, $\alpha=5$
 $5^2 \bmod 7 = 4$
- Let $X_A=3$, $Y_A = 5^3 \bmod 7 = 6$
 $5^3 \bmod 7 = 6$
- Let $X_B = 4$, $Y_B = 5^4 \bmod 7 = 2$
 $5^4 \bmod 7 = 2$
- $K_A = 2^3 \bmod 7 = 1$ $K = (Y_B)^{X_A} \bmod q$
 $5^5 \bmod 7 = 3$
- $K_B = 6^4 \bmod 7 = 1$ $K = (Y_A)^{X_B} \bmod q$
 $5^6 \bmod 7 = 1$

Cryptanalysis

- Because X_A and X_B are private, an **adversary only has** the following ingredients to work with: q , α , Y_A , and Y_B . Thus, the adversary is forced to take a discrete logarithm to determine the key. For example, to determine Bob's private key, an adversary must compute

$$X_B = \text{dlog}_{\alpha,q}(Y_B)$$

- The adversary can then calculate the key K in the same manner as Bob calculates it. That is, the adversary can calculate K as

$$K = (Y_A)^{X_B} \bmod q$$

- The security of the Diffie–Hellman key exchange lies in the fact that, while it is relatively easy to calculate exponentials modulo a prime, it is very **difficult to calculate discrete logarithms**

Here is an example. Key exchange is based on the use of the prime number $q = 353$ and a primitive root of 353, in this case $\alpha = 3$. Alice and Bob select private keys $X_A = 97$ and $X_B = 233$, respectively. Each computes its public key:

Alice computes $Y_A = 3^{97} \bmod 353 = 40$.

Bob computes $Y_B = 3^{233} \bmod 353 = 248$.

After they exchange public keys, each can compute the common secret key:

Alice computes $K = (Y_B)^{X_A} \bmod 353 = 248^{97} \bmod 353 = 160$.

Bob computes $K = (Y_A)^{X_B} \bmod 353 = 40^{233} \bmod 353 = 160$.

Digital Signatures

- **Message authentication protects** two parties who exchange messages **from any third party**.
- However, it **does not protect the two parties against each other**. Several forms of dispute between the two parties are possible.
 1. Mary may forge a different message and claim that it came from John. Mary would simply have to create a message and append an authentication code using the key that John and Mary share.
 2. John can deny sending the message. Because it is possible for Mary to forge a message, there is no way to prove that John did in fact send the message.
- There is **no complete trust between sender and receiver**, something more than authentication is needed.
- The most attractive **solution** to this problem is the **digital signature**.

Digital Signatures - Properties

- The digital signature must have the following properties:
 - It must **verify the author** and **the date and time** of the signature.
 - It must **authenticate the contents** at the time of the signature.
 - It must be **verifiable by third parties**, to resolve disputes.
- Thus, the digital signature function **includes the authentication function**.

(a) Bob signs a message

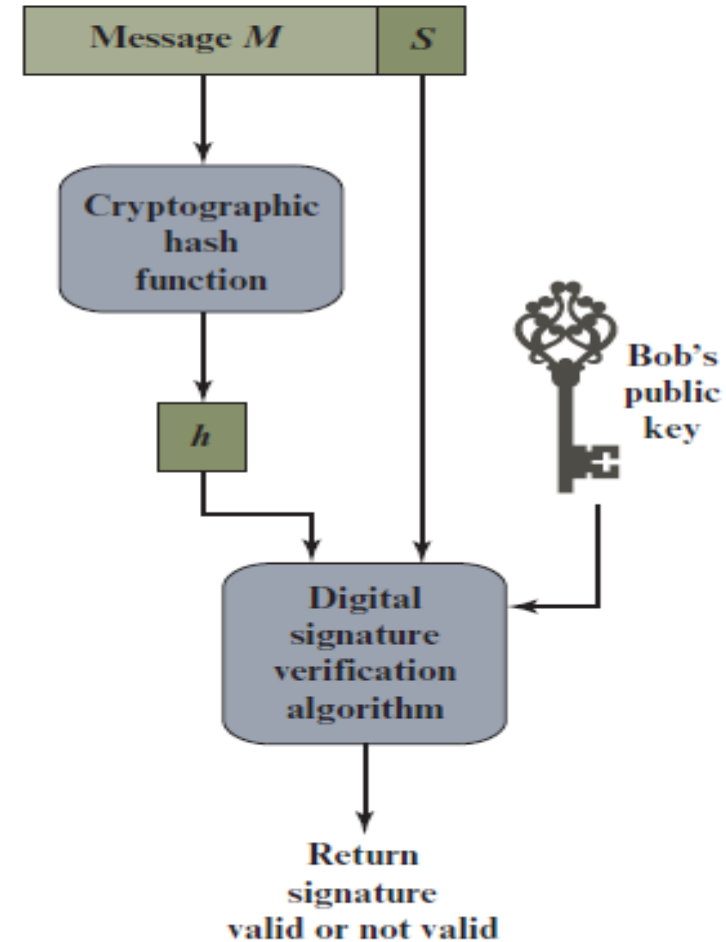
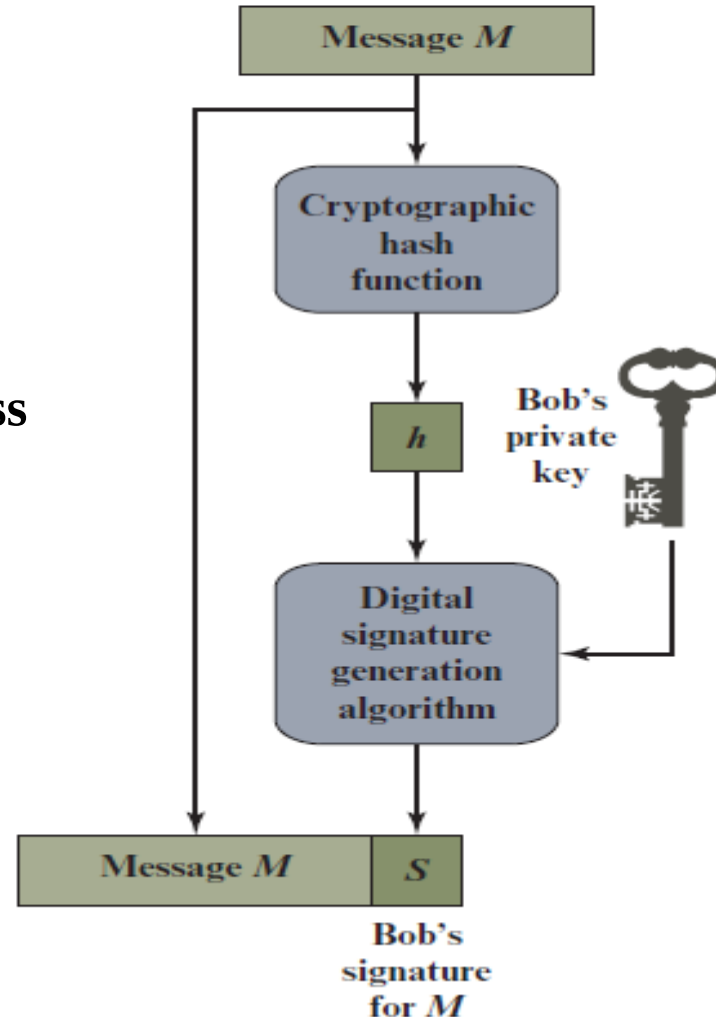


Alice



(b) Alice verifies the signature

Simplified Depiction of Essential Elements of Digital Signature Process



No one else has Bob's private key and therefore no one else could have created a signature that could be verified for this message with Bob's public key.

Digital Signature Requirements

- The **signature** must be a **bit pattern** that **depends on the message** being signed.
- The signature must use some **information only known to the sender** to prevent both forgery and denial.
- It must be relatively **easy to produce** the digital signature.
- It must be relatively **easy to recognize and verify** the digital signature.
- It must be **computationally infeasible to forge a digital signature**, either by constructing a new message for an existing digital signature or by constructing a fraudulent digital signature for a given message.
- It must be practical to **retain a copy of the digital signature** in storage.

Attacks and Forgeries

- Here **A denotes the user** whose signature method is being attacked, and **C denotes the attacker**.
- **Key-only attack:** C only knows A's public key.
- **Known message attack:** C is given access to a set of messages and their signatures.
- **Generic chosen message attack:** C chooses a list of messages before attempting to break A's signature scheme, independent of A's public key. C then obtains from A valid signatures for the chosen messages. The attack is generic, because it does not depend on A's public key; the same attack is used against everyone.
- **Directed chosen message attack:** Similar to the generic attack, except that the list of messages to be signed is chosen after C knows A's public key but before any signatures are seen.
- **Adaptive chosen message attack:** C is allowed to use A as an "oracle." This means that C may request from A signatures of messages that depend on previously obtained message-signature pairs.

Attacks and Forgeries

- Success at breaking a signature scheme as an outcome
- **Total break:** C determines A's private key.
- **Universal forgery:** C finds an efficient signing algorithm that provides an equivalent way of constructing signatures on arbitrary messages.
- **Selective forgery:** C forges a signature for a particular message chosen by C.
- **Existential forgery:** C forges a signature for at least one message. C has no control over the message. Consequently, this forgery may only be a minor nuisance to A.